

Inference is the act of using a trained model to produce an answer. In simple words, inference is the model actually doing its job, which means reading your question and writing back a reply.

Note :

An LLM does not write out the whole answer at once. It writes the answer one small piece at a time. Each small piece is called a **token**. A token is a small chunk of text, like a word or a part of a word.

So, the model generates one token, then the next token, then the next, until the full answer is complete. This is why, when you use a chat assistant, you can see the words appear one after another like someone typing. This live, token-by-token appearance is called [streaming](#).

The two phases: Prefill and Decode

Every time an LLM answers a request, it goes through two distinct phases.

- **Prefill** is the first phase. It reads and processes the whole input prompt that you sent.
- **Decode** is the second phase. It generates the output tokens one at a time.

In simple words, prefill is the model reading your question, and decode is the model writing the answer.

Both phases use the exact same model. They do not use different models or different weights. The **weights** are the millions of numbers the model learned during training.

Prefill is the phase where the model reads and processes your entire input prompt in one single pass and produces the very first output token.

Suppose you paste a question that is 1,000 tokens long. In the prefill phase, the model does not read these 1,000 tokens one by one. It reads all 1,000 tokens together, at the same time.

Let's say a student walks into an exam hall. Before writing anything, the student silently reads the entire question paper.

The student scans every question at once and jots down quick notes about what matters. This reading-the-whole-paper part is prefill.

While reading the prompt, the model also prepares some notes for later. For every input token, it computes and stores two special pieces of data, called the **Key (K)** and the **Value**

(V).

Every token is converted into three things:

- **Query (Q):** What the current token is looking for.
- **Key (K):** What each previous token offers, like a label.
- **Value (V):** The actual information that each previous token carries.

Each of these is just a list of numbers. In AI, a list of numbers is called a **vector**. So, every token has a Query vector, a Key vector, and a Value vector.

All of these stored Keys and Values together form the [KV cache](#). Do not worry about what exactly Key and Value mean right now. For now, just understand that the KV cache is the model's memory of your prompt. We will learn about it in detail soon.

So, prefill fills up this KV cache with the model's notes about every input token. And at the end of prefill, the model produces the very first output token.

Here is a very important point. **Prefill produces only the first output token, not the whole answer.** All the remaining tokens come later, in the decode phase.

Prefill is compute-bound. The compute-bound means the speed is held back by the math, nothing else. A [GPU](#) is the powerful chip that runs the model.

Inside the GPU there are **math units**, which are the parts that actually do the multiplication. During prefill, these math units are kept very busy doing heavy math.

On a high-end GPU, the math units can stay highly used during prefill, often approaching 90 percent or more on large prompts. The GPU is well used.

Why is prefill so heavy on math? Because when many tokens are processed together, the work becomes very large multiplication of big tables of numbers. A big table of numbers like this is called a matrix, and multiplying two big tables together is exactly what GPUs are built for. So, the GPU runs near its full power.

There is one more thing to know. Longer prompts mean longer prefill. A short question feels almost instant. But if we paste a huge document in front of the model, prefill takes longer, and we wait longer before the first word appears.

In fact, part of the prefill work grows even faster than the prompt length. Inside the model there is a step, called [attention](#), where every token must look at every other token to understand the context.

Attention is the way a model decides which previous tokens are important for writing the next token.

In simple words, Attention is looking back and picking what matters.

Faster prefill means sooner you see the first token.

What is Self Attention?

Self Attention is a mechanism that allows every token in a sequence to look at every other token in the same sequence, including itself, to understand the context.

In simple words, **Self Attention = Self + Attention**. The word "Attention" means each token decides how much it should focus on the other tokens.

So, every word in a sentence looks at every other word in the same sentence and figures out which words matter the most for its meaning.

Why do we need Self Attention?

The best way to learn this is by taking an example.

Consider the sentence:

| The animal did not cross the street because it was too tired.

Now, the question is: what does the word `it` refer to?

As humans, we know that `it` refers to `the animal`, because the animal was too tired to cross.

Now, let's read one more sentence:

| The animal did not cross the street because it was too wide.

Same structure. Same word `it`. But now, what does `it` refer to?

Here, `it` refers to `the street`, because the street was too wide.

We changed just one word - from `tired` to `wide` - and the meaning of `it` completely changed.

So, how did our brain figure that out? Our brain looked at the surrounding words to understand the meaning of `it`. The word `tired` pulled our attention towards `the animal`. The word `wide` pulled our attention towards `the street`.

This is exactly what happens inside the Large Language Model. Every word looks at every other word in the same sentence to understand its meaning. The model gives more attention to the words that matter and less attention to the words that do not matter. This mechanism is called **Attention**.

Now, notice that every word here is looking at the other words within the SAME sentence. The sentence is attending to itself. This is why we call it **Self Attention**.

In the first sentence, `it` pays strong attention to `animal`. In the second sentence, `it` pays strong attention to `street`. That is how the model captures the true meaning of `it` in both cases.

Without Self Attention, the word `it` would just be a generic token with no real meaning. And Self Attention is the core idea behind every modern Large Language Model.

Why Self Attention works so well

Now, let's understand why Self Attention works so well.

Before Self Attention, models like [RNN](#) and [LSTM](#) [processed tokens one by one in a sequence](#). This was slow and the connection between far-apart words became weak. [Self Attention solved both of these problems](#).

- **Parallelization:** All tokens are processed at the same time. We do not have to wait for one token to finish before processing the next. This makes training very fast on GPUs.
- **Long-range dependencies:** Every token can directly look at every other token, even if they are very far apart in the sentence. There is no decay over distance. The price of this is cost, which is why long-context models often switch to [Sliding Window Attention](#), where every token looks only at its nearby tokens.
- **Contextual understanding:** Every token gets a richer meaning based on the surrounding words. The same word can have a different representation in different sentences depending on the context.
- **Flexibility:** Self Attention does not care about the order of computation. It can handle long sequences and capture complex relationships.

That's the beauty of Self Attention.

Decode is the phase where the model generates the output tokens one at a time, reusing the KV cache that prefill prepared.

Reading the question paper was prefill. Now the student starts writing the answer, word by word. The student writes one word, thinks about it, then writes the next word, and so on. This slow, word-by-word writing is decode.

In decode, the model generates only one new token per step. Then it feeds that token back into itself to help produce the next token. Because each new token depends on the token just produced, the tokens must come strictly one after another.

This is called [autoregressive](#) generation. In simple words, the model keeps feeding its own output back as new input.

So, decode is sequential. It cannot process all the output tokens at once the way prefill processes all the input tokens at once. Each token must wait for the one before it.

At each decode step, the model does not re-read and re-process your whole prompt again. That would be very slow and wasteful.

Instead, it reads the KV cache, which is the memory that prefill already prepared.

It computes the Key and Value for just the one new token, reads all the older Keys and Values from the cache, produces the next token, and then appends the new token's Key and Value to the cache.

Decode is memory-bandwidth-bound. This is the opposite of prefill. The speed of decode is held back by how fast data can be moved from the GPU's memory into its math units, not by the math itself.

But to produce even that one token, the GPU must still read the entire model's weights and the whole growing KV cache from its memory. Remember, the weights are huge, so this is a lot of data to move. **Memory bandwidth** is the speed at which data can be moved out of the GPU's memory.

Here is the key fact to hold on to. Inside a GPU, doing the math is very fast, but fetching a large amount of data from its memory is comparatively slow.

So, prefill keeps the GPU busy with math, but decode keeps the GPU waiting on memory.

INPUT PROMPT (for example, 1,000 tokens)

```
|  
v  
+-----+  
| PREFILL (one big step) |  
| - reads ALL input tokens |  
| in parallel |  
| - writes the KV cache |  
| - GPU compute very busy |  
+-----+  
|  
| produces the FIRST output token  
v
```

+-----+ +-----+
DECODE (many small steps)	<---->	KV CACHE
- one token per step	read	(the memory of
- reads the KV cache	and	all tokens so
- appends one new entry	grow	far)
- GPU mostly waits on	+-----+	
memory		

+-----+
|
| token 2, token 3, token 4, ... token N

v
FULL ANSWER